Applications using Stack

Evaluation of Expression :

It is so complicated to evaluate an expression. We had something called BODMAS rule. **B** — Brackets (solve things inside ( ), [ ], { } first)

O — Orders (powers, square roots, or indices)

D — Division

M — Multiplication

A — Addition

S — Subtraction.

It was very simple. But in programming languages like 'C', we have multiple levels of precedence. So also we have variation of associative rules. Nesting of parentheses adds one more level of complication.

Can we have expressions which can be evaluated as soon as the operator is encountered?

In the normal way of writing, we write the operator between the operands. These are some simple examples.

3 + 4

4 * 5 + 6

4 + 5 * 6

5 * 6 + 7 * 8

These are called **infix expressions** – the operators are between the operands.

Can we have expressions where the operator appears after the operands?

Let us relook at the expressions above.

3 4 +

4 5 * 6 +

4 5 6 * +

5 6 * 7 8 * +

These are called **postfix expressions** – the operators are after the operands.

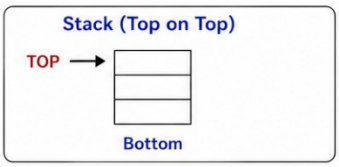
In postfix expressions, all operators at the same level of precedence. All operators are left associative. We can create a stack of operands. As soon as an operator is encountered, we pop the operands from the stack, evaluate the expression based on the operator, push the result back into the stack. When the expression is completely processed, the stack should have a single value which is the result.

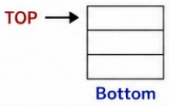
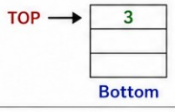
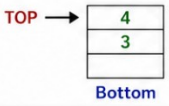
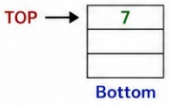
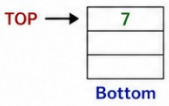
We will discuss how to convert an infix expression to postfix expression later in this document.
We will try to understand how stack is used to evaluate each of the expressions above.
These diagrams are generated using genAI tool.

Example 1 : 3 4 +

Evaluate Postfix Expression: 3 4 +

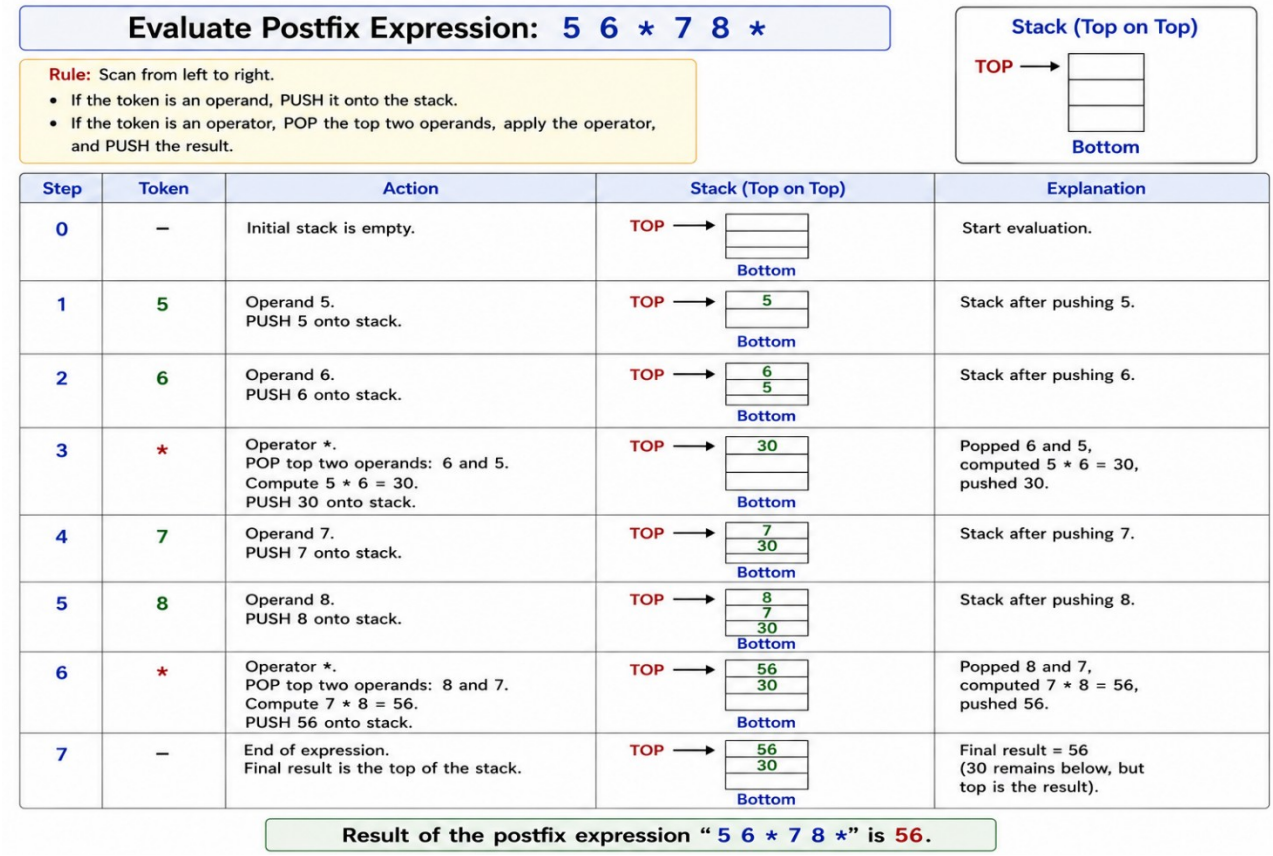
Rule: Scan from left to right.
If the token is an operand, PUSH it onto the stack.
If the token is an operator, POP the top two operands, apply the operator, and PUSH the result.



Step	Token	Action	Stack (Top on Top)	Explanation
0	–	Initial stack is empty.		Start evaluation.
1	3	Operand 3. PUSH 3 onto stack.		Stack after pushing 3.
2	4	Operand 4. PUSH 4 onto stack.		Stack after pushing 4.
3	+	Operator +. POP top two operands: 4 and 3. Compute 3 + 4 = 7. PUSH result 7 onto stack.		Popped 4 and 3, computed 3 + 4 = 7, pushed 7.
4	–	End of expression. Final result is the top of the stack.		Final result = 7

Result of the postfix expression "3 4 +" is 7.

Example 2 : 5 6 * 7 8 * +



Evaluate Postfix Expression: 5 6 * 7 8 *

Rule: Scan from left to right.

- If the token is an operand, PUSH it onto the stack.
- If the token is an operator, POP the top two operands, apply the operator, and PUSH the result.

Stack (Top on Top)

TOP → Bottom

Step	Token	Action	Stack (Top on Top)	Explanation
0	–	Initial stack is empty.	TOP → Bottom	Start evaluation.
1	5	Operand 5. PUSH 5 onto stack.	TOP → 5 / Bottom	Stack after pushing 5.
2	6	Operand 6. PUSH 6 onto stack.	TOP → 6, 5 / Bottom	Stack after pushing 6.
3	*	Operator *. POP top two operands: 6 and 5. Compute 5 * 6 = 30. PUSH 30 onto stack.	TOP → 30 / Bottom	Popped 6 and 5, computed 5 * 6 = 30, pushed 30.
4	7	Operand 7. PUSH 7 onto stack.	TOP → 7, 30 / Bottom	Stack after pushing 7.
5	8	Operand 8. PUSH 8 onto stack.	TOP → 8, 7, 30 / Bottom	Stack after pushing 8.
6	*	Operator *. POP top two operands: 8 and 7. Compute 7 * 8 = 56. PUSH 56 onto stack.	TOP → 56, 30 / Bottom	Popped 8 and 7, computed 7 * 8 = 56, pushed 56.
7	–	End of expression. Final result is the top of the stack.	TOP → 56, 30 / Bottom	Final result = 56 (30 remains below, but top is the result).

Result of the postfix expression " 5 6 * 7 8 *" is 56.

NOTE : This image has an error. No correction is made to show that these genAI tools do give wrong results. We cannot always trust them. They hallucinate!

Algorithm for evaluation of postfix expression :

We shall assume that between the operands and operators there is white space. This makes our life easier.

- Read the postfix expression

  ```
  char str[MAXSIZE + 1];
  gets(str);
  printf("input expression : :%s:\n", s);
  ```

- Create a stack of operands. We are assuming that we will support only integer arithmetic. The component of the stack is made int. Initialize the stack.

  ```
  stack_t mystack;
  init_stack(&mystack);
  ```

- Walk through the given postfix expression

  There are a few initializations. One of them is val. This is used to collect multiple digits, multiply by 10 and add to form the numeric value of the number formed by these digits.

  ```
  char *s = str;
  char ch;
  int val1; int val2;
  int val = 0;
  int is_num = 0;
  ```

  - If the character is an operator, pop as many operands as required, do the operation and push the result back into the stack.

  ```
  while( (ch = *s++) != '\0')
  {
      if(ch == '+' || ch == '-' || ch == '*' || ch == '/')
      {
          val1 = pop(&mystack);
          val2 = pop(&mystack);
          switch(ch)
          {
              case '+' : push(&mystack,val2 + val1);
  ```

```
                    break;
            case '-' : push(&mystack,val2 - val1);
                    break;
            case '*' : push(&mystack,val2 * val1);
                    break;
            case '/' : push(&mystack,val2 / val1);
                    break;
        }
    }
```

- If we encounter a digit, we are getting an operand. We compute the numeric value of one or more digits encountered.

```
    else if(ch >= '0' && ch <= '9')
    {
        is_num = 1;
        val = val * 10 + ch - '0';
    }
```

- If the number ends , then numeric value is pushed into the stack

```
    else if (ch == ' ' && is_num)
    {
        printf("pushing %d\n", val);
        push(&mystack, val);
        val = 0;
        is_num = 0;
    }
}
```

- When the input string has been completely processed, there should be one element on the stack – which is the result.

```
printf("result : %d\n", pop(&mystack));
```

Algorithm for conversion of infix to postfix:

We have discussed the advantage of having an expression in postfix. The evaluation becomes simple. There is no hierarchy. This also implies that there are no parentheses.

How do we convert an expression in infix to an expression in postfix? To do this, we use a stack of operators.

Let us discuss a few simple examples before we go through the algorithm in detail.

Example 1 :

3 + 4

Whenever we get operands, we just output them to the postfix expression.

When we encounter an operator, if the stack is empty push the operator on to the stack.

When all the operands are processed, pop from the stack into the postfix expression until the stack becomes empty.

Output 3; push + ; output 4; pop + :

Postfix expression : 3 4 +

Example 2 :

4 * 5 + 6

As per the earlier example, we do the following steps.

Output 4; push *; output 5

We encounter + in the string and we have * on the stack. Since * has a higher precedence compared to +,

pop * and push +.

Output 6

Pop the stack until empty

Pop +

Postfix expression : 4 5 * 6 +

Example 3 :

4 + 5 * 6

As per the earlier example, we do the following steps.

Output 4; push +; output 5

We encounter * in the string and we have + on the stack. Since * has a higher precedence compared to +, we push * onto the stack.

Push *

Output 6

Pop all operators from the stack

Output *

Output +

Postfix expression : 4 5 6 * +

Example 4 :

5 * 6 + 7 * 8

As per the earlier example, we do the following steps.

Output 5; push *; output 6

We encounter + in the string and we have * on the stack. Since * has a higher precedence compared to +, pop * and push +

Pop *; push +

Output 7

We encounter * in the string and we have + on the stack. Since * has a higher precedence compared to +, we push * onto the stack.

Output 8

Pop all operators

Pop *

Pop +

Postfix expression : 5 6 * 7 8 * +

Example 5 :

What if the infix expression has parentheses? There are no parentheses in postfix expression as there is no precedence.

We treat the parentheses differently. We want to always push the left parenthesis onto the stack. We will give left parenthesis a very high precedence value in the string so that it always gets pushed in. Once it is in the stack, we want all other operators to get pushed in until the right parenthesis is encountered. So, we associate a very low precedence value to the left parenthesis once it is in the stack.

When we encounter a right parenthesis, we pop all operators until we encounter left parenthesis on the stack to the output. We then pop the left parenthesis out.

3 * (4 + 5) * 6

Output 3

Push *

(in the string has very high precedence

Push (

Output 4

(in the stack has very low precedence

Push +

Output 5

Encountering } in the string

Pop until left parenthesis

Pop +

Equal precedence

If left associative, pop from the stack and push the operator in the string

Else push the operator from the string

Pop * from the stack

Push * from the string

Output 6

Pop * from the stack

Output expression : 3 4 5 + * 6 *

Algorithm:

Let us examine the algorithm:

- Create input string; read it
- Create an output string
- Provide mechanisms to walk through both

```
char in[MAXSIZE + 1]; char *s = in;
char post[MAXSIZE + 1]; char *p = post;
gets(in);
printf("input expression : :%s:\n", s);
```

- Create a stack of operators. Initialize

```
stack_t mystack;
init_stack(&mystack);
```

- Walk through the string

```
char ch;
char ch1;
while( (ch = *s++) != '\0')
{
```

  - On right parenthesis, pop all operators to output until the left parenthesis is encountered on the stack. Remove that as well.

```
if(ch == ')')
{
        while( (ch1 = pop(&mystack) ) != '('
        {
                *p++ = ' ';
                *p++ = ch1;
        }
}
```

  - On an operator, if the stack is empty, push it

```
else if(ch == '(' || ch == '+' || ch == '-' ||
        ch == '*' || ch == '/')
```

```
        {
                if(is_empty(&mystack))
                {
                        push(&mystack, ch);
                }
```

- Otherwise compare the precedence of operator in string and
  The precedence of operator in the stack
  Decide to push or pop.
  Why a loop ?

```
                else
                {
                        while(!is_empty(&mystack) &&
                                prec_in_str(ch) < prec_in_stack(peek(&mystack)))
                        {
                                *p++ = ' ';
                                *p++ = pop(&mystack);

                        }
                        push(&mystack, ch);
                }
        }
```

- Output operands directly

```
        else
        {
                *p++ = ch;
        }
}
```

Pop operators from the stack to the output

```
while(! is_empty(&mystack))
{
        *p++ = ' ';
        *p++ = pop(&mystack);
}
```

*p = '\0';

- Display the string

  printf("infix : %s\n", in);

  printf("postfix : %s\n", post);

So, we have learnt how to play with data structure as if it is another type.

Is it not?